I'm working a project in Racket: Temporal Spreadsheet. I need to build an engine that evaluates a given grid of cells, which contain either a raw number or a formula that references other cells.
ex: '(((1 1) 10) ((1 2) 20)
((2 1) (= (1 1) + (1 2)) ((2 2) (= (2 1) * 2)))
The implementation has to look up cell values and evaluate formulas, must detect cycles (circular dependencies), and must be functional (no grid mutation)
Given the above example input, how would I represent the grid as an association list of coordinates and formulas, where a coordinate is a pair (x . y)?

I have to use two primary functions: eval-cell, which handles looking up a coordinate and tracking "visited" path, and eval-formula, which handles the math.

Show less

12:15 AM

Here's how to represent the grid and implement the two primary functions in Racket:

# Grid Representation

racket

```
; Grid as association list: ((x . y) . value) pairs
(define grid
  '(((1 . 1) . 10)
    ((1 . 2) . 20)
    ((2 . 1) . (= (1 . 1) + (1 . 2)))
    ((2 . 2) . (= (2 . 1) * 2))))
```

Each entry is a `(coord . value)` pair where a coordinate is a `(x . y)` dotted pair. A value is either a raw number or a formula starting with `=`.

---

# Core Functions

racket

```
#lang racket

;;; --- Grid Lookup ---

; lookup : coord grid -> value | #f
(define (lookup coord grid)
  (let ([entry (assoc coord grid)])
    (if entry
        (cdr entry)
        (error "Cell not found" coord))))
```

```
;;; --- Cycle Detection via Visited Set ---

; eval-cell : coord grid visited -> number
; visited is a list of coords we're currently resolving (the call
stack)
(define (eval-cell coord grid visited)
  (cond
    ; Cycle detected: coord is already in the current resolution path
    [(member coord visited)
     (error "Circular dependency detected" coord)]
    [else
     (let ([value (lookup coord grid)])
       (if (number? value)
           value
           ; It's a formula — evaluate it, adding coord to visited
           (eval-formula value grid (cons coord visited))))]))


;;; --- Formula Evaluator ---

; eval-formula : formula grid visited -> number
; formula shape: (= <ref-or-num> <op> <ref-or-num>)
(define (eval-formula formula grid visited)
  (match formula
    [`(= ,lhs ,op ,rhs)
     (let ([l (resolve lhs grid visited)]
           [r (resolve rhs grid visited)])
       (apply-op op l r))]
    [_ (error "Malformed formula" formula)]))


;;; --- Resolve: number or cell reference ---

; resolve : term grid visited -> number
; A term is either a raw number or a coordinate (x . y)
(define (resolve term grid visited)
  (cond
    [(number? term) term]
    [(pair? term)   (eval-cell term grid visited)]  ; it's a coord
    [else           (error "Unknown term" term)]))


;;; --- Operator Dispatch ---
```

```
; apply-op : symbol number number -> number
(define (apply-op op l r)
  (match op
    ['+ (+ l r)]
    ['- (- l r)]
    ['* (* l r)]
    ['/ (if (zero? r)
            (error "Division by zero")
            (/ l r))]
    [_  (error "Unknown operator" op)]))
```

---

## How It Fits Together

```
eval-cell (2 . 2)
  └─ looks up → (= (2 . 1) * 2)
  └─ eval-formula, visited = [(2 . 2)]
       └─ resolve (2 . 1) → eval-cell (2 . 1), visited = [(2 . 2)]
            └─ looks up → (= (1 . 1) + (1 . 2))
            └─ eval-formula, visited = [(2 . 1) (2 . 2)]
                 └─ resolve (1 . 1) → 10
                 └─ resolve (1 . 2) → 20
                 └─ (+ 10 20) = 30
       └─ resolve 2 → 2
       └─ (* 30 2) = 60
```

The visited list is the functional cycle detector — it's threaded through every call without mutation. If `eval-cell` sees a coordinate already in `visited`, it errors immediately before any infinite loop can occur.

can you explain each function and why they work

12:15 AM

Sure! Let me walk through each function and the reasoning behind it.

---

## lookup

racket

```
(define (lookup coord grid)
  (let ([entry (assoc coord grid)])
    (if entry
        (cdr entry)
```

```
        (error "Cell not found" coord))))
```

`assoc` searches an association list for a key using `equal?` comparison. Since our coords are dotted pairs like `(1 . 1)`, `equal?` correctly matches them structurally. When a match is found, `assoc` returns the whole pair — so `(cdr entry)` unwraps it to get just the value (either a number or a formula). If nothing is found, we error immediately rather than silently returning `#f`, which would cause a confusing failure later.

---

## eval-cell

racket

```
(define (eval-cell coord grid visited)
  (cond
    [(member coord visited)
     (error "Circular dependency detected" coord)]
    [else
     (let ([value (lookup coord grid)])
       (if (number? value)
           value
           (eval-formula value grid (cons coord visited))))]))
```

This is the entry point for resolving any cell. The key idea is the `visited` list, which represents your current resolution path — every cell you're in the middle of evaluating right now. Before doing anything, it checks if `coord` is already in that path. If it is, you have a cycle: cell A needs cell B which needs cell A again, forever. Erroring here breaks the loop.

If there's no cycle, it looks up the value. A raw number needs no further work and is returned directly. A formula gets passed to `eval-formula` with `coord` consed onto visited — this is the functional "push onto the call stack" step. Crucially, visited is never mutated; each recursive call gets its own extended copy.

---

## eval-formula

racket

```
(define (eval-formula formula grid visited)
  (match formula
    [`(= ,lhs ,op ,rhs)
     (let ([l (resolve lhs grid visited)]
           [r (resolve rhs grid visited)])
       (apply-op op l r))]
```

```
    [_ (error "Malformed formula" formula)]))
```

This handles the structure of a formula. It uses `match` with a quasi-quoted pattern to destructure `(= lhs op rhs)` in one clean step — if the formula doesn't fit that shape, the wildcard `_` catches it and errors. Once destructured, it resolves both sides independently and then applies the operator. Both `resolve` calls receive the same `visited` list, which is correct — `lhs` and `rhs` are siblings, not dependencies of each other.

---

## resolve

racket

```
(define (resolve term grid visited)
  (cond
    [(number? term) term]
    [(pair? term)   (eval-cell term grid visited)]
    [else           (error "Unknown term" term)]))
```

This is a small but important bridge. Inside a formula, a term could be either a literal number like `2` or a cell reference like `(2 . 1)`. Those two cases need completely different handling, so `resolve` acts as a dispatcher. A number is already fully evaluated. A pair is a coordinate, so it hands off to `eval-cell` — which is where cycle detection and recursive lookup live. This keeps `eval-formula` clean by hiding the "what kind of thing is this term?" logic in one place.

---

## apply-op

racket

```
(define (apply-op op l r)
  (match op
    ['+ (+ l r)]
    ['- (- l r)]
    ['* (* l r)]
    ['/ (if (zero? r) (error "Division by zero") (/ l r))]
    [_ (error "Unknown operator" op)]))
```

```
Straightforward operator dispatch. It's separated into its own
function so that `eval-formula` stays focused on structure, not
arithmetic. The division case explicitly guards against `zero?`
```

because Racket would otherwise throw a low-level error with a less helpful message.

---

## Why the design works as a whole

The reason these five functions compose cleanly is that they form a **mutual recursion** with a single piece of shared state — `visited` — that is passed functionally rather than stored globally:

```
eval-cell ──calls──▶ eval-formula
                          │
                          └──calls──▶ resolve
                                        │
                                        └──calls──▶ eval-cell
                                                    (cycle closes here)
```

Because `visited` is immutable and extended by value at each step, parallel branches of the formula tree don't interfere with each other, and backtracking is free — there's nothing to undo.

how would I make a function that takes lists similar to the inputs and makes an association list out of it?

12:17 AM

Here's a function that parses your input format into the association list:

# The Parser Function

racket

```
; parse-grid : list-of-entries -> grid (association list)
; input shape: (((x y) value) ...)
; output shape: (((x . y) . value) ...)
(define (parse-grid entries)
  (map parse-entry entries))

; parse-entry : entry -> (coord . value)
(define (parse-entry entry)
  (let* ([coord-list (car entry)]   ; (x y)
         [value      (cadr entry)]  ; number or formula
         [coord      (cons (car coord-list) (cadr coord-list))]) ; (x
. y)
    (cons coord (parse-value value))))

; parse-value : value -> number or formula
```

```
(define (parse-value value)
  (cond
    [(number? value) value]
    [(and (pair? value) (eq? (car value) '=))
     (parse-formula value)]
    [else (error "Unknown value format" value)]))

; parse-formula : raw-formula -> normalized formula
; input:  (= (x y) op (x y))
; output: (= (x . y) op (x . y))
(define (parse-formula formula)
  (match formula
    [`(= ,lhs ,op ,rhs)
     `(= ,(parse-term lhs) ,op ,(parse-term rhs))]
    [_ (error "Malformed formula" formula)]))

; parse-term : term -> number or coord
(define (parse-term term)
  (cond
    [(number? term) term]
    [(pair? term)   (cons (car term) (cadr term))] ; (x y) -> (x . y)
    [else           (error "Unknown term format" term)]))
```

---

## Usage

racket

```
(define raw-input
  '(((1 1) 10)
    ((1 2) 20)
    ((2 1) (= (1 1) + (1 2)))
    ((2 2) (= (2 1) * 2))))

(define my-grid (parse-grid raw-input))

; my-grid =>
; (((1 . 1) . 10)
;  ((1 . 2) . 20)
;  ((2 . 1) = (1 . 1) + (1 . 2))
;  ((2 . 2) = (2 . 1) * 2))
```

---

## What each function does

- `parse-grid` — maps over every entry in the raw list, delegating each one to `parse-entry`. It's just a `map` wrapper.
- `parse-entry` — pulls apart one `((x y) value)` entry. It converts the flat `(x y)` coordinate list into a proper dotted pair `(x . y)` using `cons`, then delegates the value to `parse-value`.
- `parse-value` — decides whether the value is a plain number (pass it through) or a formula starting with `=` (send it to `parse-formula`).
- `parse-formula` — uses `match` to destructure the formula and converts both terms from `(x y)` list style to `(x . y)` dotted pair style via `parse-term`.
- `parse-term` — handles the two cases that can appear inside a formula: a raw number (returned as-is) or a coordinate list `(x y)` that needs to become `(x . y)`.

The key transformation happening throughout is `(x y)` → `(x . y)` — your input uses proper two-element lists for coordinates, but the engine expects dotted pairs so that `assoc` lookups work correctly.

where is the assoc function defined?

12:53 PM

`assoc` is a built-in Racket function — it comes from the standard library, so you don't need to define it yourself.

It's part of the base language and is available in `#lang racket` without any imports. It works on any association list and uses `equal?` for comparison by default:

racket

```
(assoc '(1 . 1) '(((1 . 1) . 10) ((1 . 2) . 20)))
; => ((1 . 1) . 10)

(assoc '(9 . 9) '(((1 . 1) . 10) ((1 . 2) . 20)))
; => #f
```

Racket also provides two variants if you ever need them:

- `assv` — uses `eqv?` for comparison (good for numbers and symbols)
- `assq` — uses `eq?` for comparison (good for symbols and identical objects)

For our grid, `assoc` is the right choice because it uses `equal?`, which does structural comparison — meaning it compares the contents of the dotted pair `(1 . 1)` rather than checking if they're the exact same object in memory. `assq` would fail here because two separately constructed `(1 . 1)` pairs wouldn't be `eq?` to each other even if they look identical.

in the second cond of parse-value function why is pair? function used instead of list?

1:16 PM

Great question! Both would actually work here, but `pair?` is the more accurate choice for this specific case. Here's why:

## What each predicate checks

- `pair?` — returns `#t` for anything constructed with `cons`, meaning it has a `car` and a `cdr`. This includes both proper lists and dotted pairs.
- `list?` — returns `#t` only for proper lists that end in `'()`, and it traverses the entire list to verify this.

## In our case

The formula coming in looks like:

racket

```
'(= (1 1) + (1 2))
```

This is a proper list, so `list?` would return `#t` here too. So functionally, either works for this input.

## Why `pair?` is still the better choice

1. We only care about the first element. We're just checking `(eq? (car value) '=)` — we don't care about the rest of the structure. `pair?` expresses exactly that: "does this have a `car` I can look at?"

2. `list?` is more expensive. It walks the entire list to confirm it ends in `'()`. For just checking the head element, that's wasted work.

3. `pair?` is more honest about intent. Using `list?` would imply you care that the input is a well-formed list all the way through, which you don't — you're about to hand it to `parse-formula` which uses `match` to validate the structure properly anyway.

A good rule of thumb in Racket is:

racket

```
; Use pair? when you just need to destructure (car/cdr access)
; Use list? when you need to guarantee proper list structure
```

What would I need to change for returning '#CYCLE if the lookup enters a path it has already visited

1:36 PM

Just change the `error` call in `eval-cell` to return `'#CYCLE` instead:

racket

```
(define (eval-cell coord grid visited)
  (cond
    [(member coord visited)
     '#CYCLE]                          ; return symbol instead of
erroring
    [else
     (let ([value (lookup coord grid)])
       (if (number? value)
           value
           (eval-formula value grid (cons coord visited))))]))
```

However, you also need to update `eval-formula` and `apply-op` to propagate `'#CYCLE` instead of trying to do math with it:

racket

```
(define (eval-formula formula grid visited)
  (match formula
    [`(= ,lhs ,op ,rhs)
     (let ([l (resolve lhs grid visited)]
           [r (resolve rhs grid visited)])
       (if (or (eq? l '#CYCLE) (eq? r '#CYCLE))
           '#CYCLE                       ; propagate up if either side
is a cycle
           (apply-op op l r)))]
    [_ (error "Malformed formula" formula)]))
```

`apply-op` itself doesn't need to change since the check happens before it's ever called.

---

# Why propagation matters

Without the check in `eval-formula`, a cycle in a referenced cell would return `'#CYCLE` as a value that then gets passed into `apply-op` like:

racket

```
(+ '#CYCLE 10)  ; crashes — #CYCLE is not a number
```

With the propagation check, `'#CYCLE` bubbles up cleanly through the whole call chain, so the final result of any cell that depends on a cycle also returns `'#CYCLE` rather than crashing.

how do I fix syntax error from '#

1:40 PM

The issue is that `'#CYCLE` is not valid Racket syntax. The `#` character has special meaning in Racket. Just drop the `#` and use a plain symbol instead:

racket

```
'CYCLE
```

So the changes are:

racket

```
(define (eval-cell coord grid visited)
  (cond
    [(member coord visited)
     'CYCLE]
    [else
     (let ([value (lookup coord grid)])
       (if (number? value)
           value
           (eval-formula value grid (cons coord visited))))]))

(define (eval-formula formula grid visited)
  (match formula
    [`(= ,lhs ,op ,rhs)
     (let ([l (resolve lhs grid visited)]
           [r (resolve rhs grid visited)])
       (if (or (eq? l 'CYCLE) (eq? r 'CYCLE))
           'CYCLE
           (apply-op op l r)))]
    [_ (error "Malformed formula" formula)]))
```

In Racket, `#` is reserved for special literals like `#t`, `#f`, `#\a` (characters), `#(...)` (vectors), etc.

A plain quoted symbol like `'CYCLE` or `'ERROR` is the idiomatic way to return a sentinel value in Racket.

the project specifications say to return '#CYCLE

1:50 PM

Then the fix is to make it a string instead of a symbol, which avoids the `#` syntax issue:

racket

```
"#CYCLE"
```

Or alternatively, your spec might mean a quoted list:

racket

```
'(CYCLE)
```

But if it must literally be `#CYCLE` as a symbol, you can use the pipe notation to escape it:

racket

```
'|#CYCLE|
```

Pipe characters `|...|` let you create symbols with otherwise illegal characters in them. So the code would look like:

racket

```
(define (eval-cell coord grid visited)
  (cond
    [(member coord visited)
     '|#CYCLE|]
    [else
     (let ([value (lookup coord grid)])
       (if (number? value)
           value
           (eval-formula value grid (cons coord visited))))]))

(define (eval-formula formula grid visited)
  (match formula
    [`(= ,lhs ,op ,rhs)
     (let ([l (resolve lhs grid visited)]
           [r (resolve rhs grid visited)])
       (if (or (eq? l '|#CYCLE|) (eq? r '|#CYCLE|))
           '|#CYCLE|
           (apply-op op l r)))]
    [_ (error "Malformed formula" formula)]))
```

I'd recommend checking with your project spec whether they mean a string `"#CYCLE"`, a plain symbol `'CYCLE`, or truly the escaped symbol `'|#CYCLE|`, since all three are valid Racket values but behave differently when compared.

how do I make it so the program evaluates the grid so every formula is replaced by its numeric result

1:57 PM

You need a function that maps over the grid and evaluates every cell:

racket

```
; eval-grid : grid -> grid
; returns a new grid with all formulas replaced by their numeric
results
(define (eval-grid grid)
  (map (lambda (entry)
         (let* ([coord (car entry)]
                [value (eval-cell coord grid '())])
           (cons coord value)))
       grid))
```

# Usage

racket

```
(define raw-input
  '(((1 1) 10)
    ((1 2) 20)
    ((2 1) (= (1 1) + (1 2)))
    ((2 2) (= (2 1) * 2))))

(define my-grid (parse-grid raw-input))
(define evaluated-grid (eval-grid my-grid))

; evaluated-grid =>
; (((1 . 1) . 10)
;  ((1 . 2) . 20)
;  ((2 . 1) . 30)
;  ((2 . 2) . 60))
```

# How it works

It maps over every entry in the grid, keeping the coordinate the same but replacing the value by calling `eval-cell` on it. Each call to `eval-cell` starts with an empty `visited` list `'()` since we're beginning a fresh evaluation for each cell. The original grid is always passed as the lookup source, so cross-cell references still resolve correctly even as we build the new one.

what would I change for if a formula refers to a cell that is undefined treat the empty cell as 0

2:07 PM

Just change the `error` in `lookup` to return `0` instead:

racket

```
(define (lookup coord grid)
  (let ([entry (assoc coord grid)])
    (if entry
        (cdr entry)
        0)))
```

That's the only change needed. Since `lookup` is the single place where cell values are retrieved, returning `0` there automatically propagates through `eval-cell`, `resolve`, and `eval-formula` without any other modifications — they all just see a `0` and treat it as a normal number.

Since parser checks if formulas are valid, can I remove the error case in match during eval-formula function

2:08 PM

Technically yes, but it's not recommended. Here's why:

## What the parser catches

The parser validates formulas coming from your raw input, so any formula entered through `parse-grid` will be well-formed by the time it reaches `eval-formula`.

## Why you should keep the error case anyway

1. Defensive programming. If you ever call `eval-formula` directly in testing or from another part of the code that bypasses the parser, you'll get a cryptic Racket match failure instead of a clear message:

racket

```
; Without the wildcard _ case, this gives a confusing error:
match: no matching clause for (= (1 . 1) + (1 . 2) extraterm)

; With it, you get your own clear message:
; "Malformed formula (= (1 . 1) + (1 . 2) extraterm)"
```

2. The parser and evaluator are separate concerns. The parser guarantees its own output is valid, but `eval-formula` doesn't know who called it. Keeping the error case means `eval-formula` is independently correct regardless of where it's called from.

3. It costs nothing. It's one line and makes the code more robust.

So while you *can* remove it and the code will work fine for your current use case, the general principle in Racket and functional programming is to make each function defensive about its own inputs rather than relying on callers to guarantee correctness.

how would I implement a distance tax, which is the manhattandistance between two cells that reference eachother

2:28 PM

Here's how to implement a Manhattan distance tax:

## Manhattan Distance Function

racket

```
; manhattan-distance : coord coord -> number
(define (manhattan-distance c1 c2)
  (+ (abs (- (car c1) (car c2)))
     (abs (- (cdr c1) (cdr c2)))))
```

## Updated eval-formula with Tax

You need to pass the current cell's coordinate into `eval-formula` so it knows where it's being called from:

racket

```
; eval-cell passes its own coord to eval-formula
(define (eval-cell coord grid visited)
  (cond
    [(member coord visited)
     '|#CYCLE|]
    [else
     (let ([value (lookup coord grid)])
       (if (number? value)
           value
           (eval-formula value grid (cons coord visited) coord)))]))
; pass coord

; eval-formula applies the distance tax on each cell reference
resolved
(define (eval-formula formula grid visited current-coord)
  (match formula
    [`(= ,lhs ,op ,rhs)
     (let ([l (resolve lhs grid visited current-coord)]
           [r (resolve rhs grid visited current-coord)])
       (if (or (eq? l '|#CYCLE|) (eq? r '|#CYCLE|))
           '|#CYCLE|
           (apply-op op l r)))]
    [_ (error "Malformed formula" formula)]))

; resolve applies the tax when the term is a cell reference
(define (resolve term grid visited current-coord)
  (cond
    [(number? term) term]
    [(pair? term)
     (let ([value (eval-cell term grid visited)])
       (if (eq? value '|#CYCLE|)
           '|#CYCLE|
           (+ value (manhattan-distance current-coord term))))]  ;
tax applied here
    [else (error "Unknown term" term)]))
```

# Example

racket

```
; (2 . 1) references (1 . 1) and (1 . 2)
```

```
; distance from (2.1) to (1.1) = |2-1| + |1-1| = 1
; distance from (2.1) to (1.2) = |2-1| + |1-2| = 2
; so (2 . 1) = (10 + 1) + (20 + 2) = 33  instead of 30

; (2 . 2) references (2 . 1)
; distance from (2.2) to (2.1) = |2-2| + |2-1| = 1
; so (2 . 2) = (33 + 1) * 2 = 68  instead of 60
```

# What changed and why

- `manhattan-distance` computes the grid distance between two coordinates using the standard formula — sum of absolute differences in x and y.
- `eval-cell` now passes its own `coord` down to `eval-formula` so it knows where the formula lives.
- `eval-formula` forwards `current-coord` to `resolve` for both terms.
- `resolve` is where the tax is actually applied — after getting a cell's value, it adds the distance between the current cell and the referenced cell. This is the right place because `resolve` is the only function that knows both the reference target (`term`) and can receive the source (`current-coord`).

would the following changes work as a parity check, where a cell that evaluates to an even value can only reference other even cells, and returns 'parity-error if it tries to reference an odd cell?

```
; eval-formula : formula grid visited -> number
; formula shape: (= <ref-or-num> <op> <ref-or-num>)
(define (eval-formula formula grid visited current-coord)
  (match formula
    [`(= ,lhs ,op ,rhs)
     (let ([l (resolve lhs grid visited current-coord)]
           [r (resolve rhs grid visited current-coord)])
       (cond
         [(or (eq? l '|#CYCLE|) (eq? r '|#CYCLE|)) 'cycle-error] ;propagate up if either side is a cycle
         [(or (eq? l '|#PARITY|) (eq? r '|#PARITY|)) 'parity-error] ;propagate up if either side has parity error
         [else (apply-op op l r)]))]
    [_ 'Malformed-formula]))
;;; --- Resolve: number or cell reference ---
; resolve : term grid visited -> number
; A term is either a raw number or a coordinate (x . y)
(define (resolve term grid visited current-coord)
  (cond
    [(number? term) term]
    [(pair? term)  ; it's a coord
     (let ([term-value (eval-cell term grid visited)]
```

```
      [coord-value (eval-cell current-coord grid visited)])
    (cond
     [(eq? term-value '|#CYCLE|) '|#CYCLE|]
     [(and (= (modulo term-value 2) 0) (not (= (modulo coord-value 2) 0)))
      '|#PARITY|]
     [else (- term-value (manhattan-distance current-coord term))]))]
  [else        'Unknown-term]))
```

Show more

3:23 PM

The logic has the parity check backwards, and there are a couple of other issues:

# Problems

1. Parity check is inverted. Your condition checks if `term-value` is even and `coord-value` is odd, but the spec says an even *result* can only reference even *cells*. The check should be the other way — if `coord-value` is even and `term-value` is odd, that's the error:

racket

```
; Wrong: checks if referenced cell is even and current is odd
[(and (= (modulo term-value 2) 0) (not (= (modulo coord-value 2)
0)))]

; Correct: checks if current cell is even but referenced cell is odd
[(and (= (modulo coord-value 2) 0) (not (= (modulo term-value 2)
0)))]
```

2. `coord-value` calls `eval-cell` on the current cell inside its own resolution. This risks triggering `'|#CYCLE|` on itself since the current coord is already in `visited`. You should pass the final evaluated result down instead.

3. Propagation symbols are inconsistent. `eval-formula` checks for `'|#PARITY|` but returns `'parity-error`, and checks for `'|#CYCLE|` but returns `'cycle-error`. Pick one symbol per error and use it everywhere.

# Fixed Version

racket

```
; eval-cell passes its evaluated numeric result to eval-formula
(define (eval-cell coord grid visited)
  (cond
    [(member coord visited) '|#CYCLE|]
    [else
```

```
      (let ([value (lookup coord grid)])
        (if (number? value)
            value
            (eval-formula value grid (cons coord visited) coord)))]))

; eval-formula passes the resolved numeric result of current cell
down
(define (eval-formula formula grid visited current-coord)
  (match formula
    [`(= ,lhs ,op ,rhs)
     (let* ([l (resolve lhs grid visited current-coord)]
            [r (resolve rhs grid visited current-coord)]
            [result (cond
                      [(or (eq? l '|#CYCLE|)  (eq? r '|#CYCLE|))
'|#CYCLE|]
                      [(or (eq? l '|#PARITY|) (eq? r '|#PARITY|))
'|#PARITY|]
                      [else (apply-op op l r)])])
       result)]
    [_ (error "Malformed formula" formula)]))

; resolve receives the already-evaluated current cell value to avoid
re-evaluation
(define (resolve term grid visited current-coord)
  (cond
    [(number? term) term]
    [(pair? term)
     (let* ([term-value  (eval-cell term grid visited)]
            [coord-value (eval-cell current-coord grid (remove
current-coord visited))])
       (cond
         [(eq? term-value  '|#CYCLE|)  '|#CYCLE|]
         [(eq? coord-value '|#CYCLE|)  '|#CYCLE|]
         [(eq? coord-value '|#PARITY|) '|#PARITY|]
         ; even current cell referencing an odd cell = parity error
         [(and (even? coord-value) (odd? term-value)) '|#PARITY|]
         [else (+ term-value (manhattan-distance current-coord
term))]))]
    [else (error "Unknown term" term)]))
```

# Key fixes summary

- Parity condition flipped to `(even? coord-value) (odd? term-value)`

- Using Racket's built-in `even?` and `odd?` instead of `modulo` comparisons, which is cleaner
- Consistent use of `'|#CYCLE|` and `'|#PARITY|` everywhere for propagation
- Removed the re-evaluation risk by using `remove` to temporarily exclude `current-coord` from visited when evaluating it, avoiding false cycle detection on itself

the actual evaluation should be 'cycle-error, but '#CYCLE should be used at intermediate stage

3:28 PM

Good distinction! `'|#CYCLE|` is the internal sentinel that propagates through the evaluation, and `'cycle-error` is the final user-facing result. Same idea for parity. You just need to translate at the end in `eval-cell`:

racket

```
(define (eval-cell coord grid visited)
  (cond
    [(member coord visited) '|#CYCLE|]  ; internal sentinel
    [else
     (let ([value (lookup coord grid)])
       (if (number? value)
           value
           (let ([result (eval-formula value grid (cons coord
visited) coord)])
             (cond
               [(eq? result '|#CYCLE|)  'cycle-error]   ; translate
at the surface
               [(eq? result '|#PARITY|) 'parity-error]  ; translate
at the surface
               [else result])))))]))
```

The key idea is that `'|#CYCLE|` and `'|#PARITY|` are only ever seen inside `eval-formula` and `resolve` as they bubble up. By the time `eval-cell` returns to the outside world — including to `eval-grid` — it translates them into the clean user-facing symbols. Nothing outside `eval-cell` ever needs to know about the internal sentinels.

when given this grid: '(((1 . 1) . 10) ((1 . 2) = (1 . 1) + 0) ((2 . 2) = (1 . 2) * (1 . 1)))
eval-grid fails. Explain why

3:48 PM

The issue is that your grid is not properly formatted. The formula entries are missing the outer `cons` structure. Let's compare:

# What your grid looks like

racket

```
'(((1 . 1) . 10)
  ((1 . 2) = (1 . 1) + 0)       ; wrong - this is a flat list of 5
elements
  ((2 . 2) = (1 . 2) * (1 . 1))) ; wrong - this is a flat list of 5
elements
```

## What it should look like

racket

```
'(((1 . 1) . 10)
  ((1 . 2) . (= (1 . 1) + 0))       ; correct - coord dotted with
formula
  ((2 . 2) . (= (1 . 2) * (1 . 1)))) ; correct - coord dotted with
formula
```

## Why it fails

When `lookup` does `(cdr entry)` on `((1 . 2) = (1 . 1) + 0)`, it gets `(= (1 . 1) + 0)` — but only by accident since `cdr` of a flat list returns the rest of the list. Then `parse-value` receives `(= (1 . 1) + 0)` which looks correct, but the coordinates inside are still flat lists `(1 . 1)` which in this context are actually dotted pairs already so that part works. The real issue is that `assoc` uses `equal?` to match the key, and with a flat list structure the `car` of each entry is `(1 . 2)` which is actually the whole sublist, causing mismatches.

The fix is to either use `parse-grid` to build your grid from raw input, or write the association list correctly with dots:

racket

```
; Use parse-grid from raw input (recommended)
(define my-grid
  (parse-grid '(((1 1) 10)
                ((1 2) (= (1 1) + 0))
                ((2 2) (= (1 2) * (1 1))))))

; Or write the alist directly with correct dotted structure
(define my-grid
  '(((1 . 1) . 10)
    ((1 . 2) . (= (1 . 1) + 0))
    ((2 . 2) . (= (1 . 2) * (1 . 1)))))
```

that grid is the result of calling parse-grid on `( ((1 1) 10) ((1 2) (= (1 1) + 0)) ((2 2) (= (1 2) * (1 1))) )

3:50 PM

That means the bug is in `parse-entry`. Let's trace through what it does with `((1 2) (= (1 1) + 0))`:

racket

```
(define (parse-entry entry)
  (let* ([coord-list (car entry)]   ; (1 2)
         [value      (cadr entry)]  ; (= (1 1) + 0)
         [coord      (cons (car coord-list) (cadr coord-list))]) ; (1 . 2)
    (cons coord (parse-value value))))
```

`(cons coord (parse-value value))` produces:

racket

```
(cons '(1 . 2) '(= (1 1) + 0))
; => ((1 . 2) = (1 1) + 0)   ; WRONG - flat list, not a dotted pair
```

# The fix

You need to wrap `parse-value` in another `cons` or use a dotted pair:

racket

```
(define (parse-entry entry)
  (let* ([coord-list (car entry)]
         [value      (cadr entry)]
         [coord      (cons (car coord-list) (cadr coord-list))])
    (cons coord (parse-value value))))  ; wrong
    ; should be:
    (cons coord . (parse-value value))  ; still wrong, this is not valid syntax
```

The correct fix is to wrap the value in a `list` and use `cons` properly:

racket

```
(define (parse-entry entry)
  (let* ([coord-list (car entry)]
         [value      (cadr entry)]
         [coord      (cons (car coord-list) (cadr coord-list))])
    (cons coord (parse-value value))))
```

Wait — `cons` of two things already makes a dotted pair. The real bug is in `parse-formula` not wrapping the result correctly:

racket

```
; parse-formula returns a list, not a dotted pair formula
```

```
(define (parse-formula formula)
  (match formula
    [`(= ,lhs ,op ,rhs)
     `(= ,(parse-term lhs) ,op ,(parse-term rhs))]  ; this is a list
    [_ (error "Malformed formula" formula)]))
```

The quasiquote `` `(= ...) `` builds a proper list, which is correct for the formula itself. The actual bug is that `cons` of a coord and a list flattens it:

racket

```
(cons '(1 . 2) '(= (1 . 1) + 0))
; => ((1 . 2) = (1 . 1) + 0)  ; flat list, not ((1 . 2) . (= ...))
```

# Actual fix

Wrap the parsed value in a `list` before consing, or use `list` instead of `cons` in `parse-entry` and adjust `lookup` accordingly. The simplest fix is to wrap the value:

racket

```
(define (parse-entry entry)
  (let* ([coord-list (car entry)]
         [value      (cadr entry)]
         [coord      (cons (car coord-list) (cadr coord-list))])
    (list coord (parse-value value))))  ; use list instead of cons
```

And update `lookup` to use `cadr` instead of `cdr`:

racket

```
(define (lookup coord grid)
  (let ([entry (assoc coord grid)])
    (if entry
        (cadr entry)   ; cadr instead of cdr
        0)))
```

This way the grid entries are `((1 . 2) (= (1 . 1) + 0))` — a two element list — rather than trying to force a dotted pair structure that `cons` flattens.

this fix with the same input resulted in an infinite loop

3:53 PM

The infinite loop is coming from `resolve` calling `eval-cell` on `current-coord` to get `coord-value` for the parity check — but `current-coord` is itself still being evaluated, so it recurses forever.

The fix is to not re-evaluate `current-coord` inside `resolve`. Instead, pass the evaluated result of the current cell down from `eval-formula`:

racket

```
; eval-formula evaluates both terms and passes its own result for
parity checking
(define (eval-formula formula grid visited current-coord)
  (match formula
    [`(= ,lhs ,op ,rhs)
     (let* ([l (resolve lhs grid visited current-coord)]
            [r (resolve rhs grid visited current-coord)])
       (cond
         [(or (eq? l '|#CYCLE|)  (eq? r '|#CYCLE|))  '|#CYCLE|]
         [(or (eq? l '|#PARITY|) (eq? r '|#PARITY|)) '|#PARITY|]
         [else
          (let ([result (apply-op op l r)])
            ; check parity of final result here, after we know it
            (if (and (number? result)
                     (even? result)
                     (or (and (number? l) (odd? l))
                         (and (number? r) (odd? r))))
                '|#PARITY|
                result))]))]
    [_ (error "Malformed formula" formula)]))

; resolve no longer needs to evaluate current-coord at all
(define (resolve term grid visited current-coord)
  (cond
    [(number? term) term]
    [(pair? term)
     (let ([term-value (eval-cell term grid visited)])
       (cond
         [(eq? term-value '|#CYCLE|)  '|#CYCLE|]
         [(eq? term-value '|#PARITY|) '|#PARITY|]
         [else (+ term-value (manhattan-distance current-coord
term))]))]
    [else (error "Unknown term" term)]))
```

The key change is that the parity check now happens in `eval-formula` after the result is computed, since that's the only point where you actually know what the current cell's value will be — you can't know it before the formula is done evaluating.

I need parity guard where the engine checks if an even cell is attempting to work with an odd cell before it performs an operation

4:00 PM

Move the parity check into `apply-op` so it guards before performing the operation:

racket

```
(define (apply-op op l r)
  (cond
    [(and (even? l) (odd? r)) '|#PARITY|]   ; guard: even cell
working with odd cell
    [(and (even? r) (odd? l)) '|#PARITY|]   ; guard: symmetric case
    [else
     (match op
       ['+ (+ l r)]
       ['- (- l r)]
       ['* (* l r)]
       ['/ (if (zero? r)
               (error "Division by zero")
               (/ l r))]
       [_ (error "Unknown operator" op)])]))
```

And simplify `eval-formula` back to just propagating the sentinel:

racket

```
(define (eval-formula formula grid visited current-coord)
  (match formula
    [`(= ,lhs ,op ,rhs)
     (let* ([l (resolve lhs grid visited current-coord)]
            [r (resolve rhs grid visited current-coord)])
       (cond
         [(or (eq? l '|#CYCLE|)  (eq? r '|#CYCLE|))  '|#CYCLE|]
         [(or (eq? l '|#PARITY|) (eq? r '|#PARITY|)) '|#PARITY|]
         [else (apply-op op l r)]))]
    [_ (error "Malformed formula" formula)]))
```

This is the right place for the guard because `apply-op` is the exact moment the two values are about to interact — it has both operands available and sits just before the arithmetic happens, so it can reject the operation cleanly before any computation occurs.

now getting contract violation in manhattan-distance function

4:20 PM

The contract violation is happening because `resolve` calls `manhattan-distance` with `current-coord` and `term`, but `term` at that point is a raw list `(1 2)` from the input — not a dotted pair `(1 . 2)`. So when `manhattan-distance` does `(cdr c)` it gets `(2)` instead of `2`.

The fix is to parse `term` into a dotted pair before passing it to `manhattan-distance`:

racket

```
(define (resolve term grid visited current-coord)
  (cond
    [(number? term) term]
    [(pair? term)
```

```
      (let* ([coord (cons (car term) (cadr term))]  ; normalize (1 2)
-> (1 . 2)
             [term-value (eval-cell coord grid visited)])
        (cond
          [(eq? term-value '|#CYCLE|)  '|#CYCLE|]
          [(eq? term-value '|#PARITY|) '|#PARITY|]
          [else (+ term-value (manhattan-distance current-coord
coord))]))]
    [else (error "Unknown term" term)]))
```

This happens because `parse-formula` uses `parse-term` to convert coordinates, but since we switched `parse-entry` to use `list` instead of `cons`, the formula coordinates are still coming through as flat lists `(1 2)` rather than dotted pairs `(1 . 2)`. Normalizing inside `resolve` is the safest fix since it handles the conversion at the point of use.

I want to rewrite the parsing section completely

4:22 PM

Here's a clean rewrite of the parsing section:

racket

```
;;; --- Parsing ---

; coord : list -> (x . y)
(define (parse-coord c)
  (cons (car c) (cadr c)))

; parse-term : term -> number or (x . y)
(define (parse-term t)
  (if (number? t)
      t
      (parse-coord t)))

; parse-formula : raw-formula -> normalized formula
; input:  (= (x y) op (x y))
; output: (= (x . y) op (x . y))
(define (parse-formula f)
  (match f
    [`(= ,lhs ,op ,rhs)
     (list '= (parse-term lhs) op (parse-term rhs))]
    [_ (error "Malformed formula" f)]))

; parse-value : value -> number or formula
(define (parse-value v)
  (if (number? v)
      v
```

```
      (parse-formula v)))

; parse-entry : raw-entry -> (coord . value)
(define (parse-entry e)
  (cons (parse-coord (car e))
        (parse-value (cadr e))))

; parse-grid : raw-grid -> grid
(define (parse-grid g)
  (map parse-entry g))
```

# What changed

The key differences from the previous version are:

- `parse-entry` now uses `cons` correctly — `(cons coord value)` produces a proper dotted pair `((x . y) . value)` so `lookup` can use `cdr` again instead of `cadr`.
- `parse-formula` uses `list` explicitly instead of quasiquote, making it clear exactly what structure is being built.
- `parse-coord` is pulled into its own named function used consistently everywhere, so coordinates are always converted to dotted pairs in one place.
- Every function has a single clear responsibility with no overlap, making the flow `raw input → parse-grid → parse-entry → parse-coord/parse-value → parse-formula → parse-term` straightforward to follow.

And revert `lookup` back to using `cdr`:

racket

```
(define (lookup coord grid)
  (let ([entry (assoc coord grid)])
    (if entry
        (cdr entry)
        0)))
```

rewrite eval-grid with this new logic in mind

4:39 PM

racket

```
;;; --- Grid Evaluation ---

; eval-grid : grid -> grid
; returns a new grid with all formulas replaced by their results
(define (eval-grid grid)
  (map (lambda (entry)
         (cons (car entry)
               (eval-cell (car entry) grid '())))
```

```
       grid))
```

That's all that needs to change. Since `parse-entry` now correctly produces `((x . y) . value)` dotted pairs, `(car entry)` gets the coord and `(cdr entry)` gets the value — and `eval-cell` handles all the evaluation logic including cycle detection, parity, and formula resolution.

## Full evaluation flow with the new parsing

```
raw input
    └─▶ parse-grid
            └─▶ parse-entry  →  ((x . y) . value)
                                        │
                                   eval-grid
                                        └─▶ eval-cell (car entry)
                                                └─▶ lookup  →
number or formula
                                                └─▶ eval-formula
                                                        └─▶
resolve

└─▶ apply-op
                                                └─▶ translate
sentinel to user-facing symbol
```

there is first: contract violation in eval-grid with given being passed as: '((1 . 1) . 10)

4:47 PM

The contract violation is happening because `eval-cell` is being called with `(car entry)` which is the coord, but `eval-cell` then calls `lookup` which calls `assoc` — and `assoc` is failing because the grid entries are now `((x . y) . value)` dotted pairs, so `assoc` can't traverse them correctly.

Let's trace through what `assoc` sees:

racket

```
; entry = ((1 . 1) . 10)
; (car entry) = (1 . 1)  <- coord, correct
; (cdr entry) = 10       <- value, correct

; but assoc traverses the grid looking at (car each-entry):
; (car '((1 . 1) . 10)) = (1 . 1)  <- this is fine
; equal? '(1 . 1) '(1 . 1) = #t    <- this should match
```

The actual issue is that `eval-cell` receives the coord and passes it to `lookup`, but inside `eval-formula`, `resolve` is calling `eval-cell` with the parsed term which is already a dotted pair `(1 . 1)` — and `(pair? term)` in `resolve` is matching it as a coordinate when it might be matching the entire entry instead. Trace through `resolve`:

racket

```
; term = (1 . 1)  <- a dotted pair
; (pair? term) = #t  <- matches coord branch, correct
; (cons (car term) (cadr term))  <- BUG: cadr of a dotted pair (1 .
1) is WRONG
;   (car '(1 . 1))  = 1
;   (cadr '(1 . 1)) = ERROR - dotted pairs have no cadr!
```

`cadr` is the bug — it only works on proper lists. Since `parse-term` already converts to dotted pairs, `resolve` should use the term directly:

racket

```
(define (resolve term grid visited current-coord)
  (cond
    [(number? term) term]
    [(pair? term)
     (let ([term-value (eval-cell term grid visited)])  ; term is
already (x . y)
       (cond
         [(eq? term-value '|#CYCLE|)  '|#CYCLE|]
         [(eq? term-value '|#PARITY|) '|#PARITY|]
         [else (+ term-value (manhattan-distance current-coord
term))]))]
    [else (error "Unknown term" term)]))
```

Since `parse-term` already produces proper dotted pair coords, `resolve` can trust that any `pair?` term is already a valid `(x . y)` coord and pass it directly to `eval-cell` without any further conversion.

the program is not getting past the eval-grid function when I call:
(define my-grid (parse-grid `( ((1 1) 10) ((1 2) (= (1 1) + 0)) ((2 2) (= (1 2) * (1 1))) )))
(define new-grid (eval-grid my-grid))

4:53 PM

Let's add some debug traces to see exactly where it's getting stuck:

racket

```
(define (eval-grid grid)
  (display "Starting eval-grid") (newline)
  (display "Grid: ") (display grid) (newline)
  (map (lambda (entry)
```

```
          (display "Evaluating entry: ") (display entry) (newline)
          (let ([coord (car entry)])
            (display "Coord: ") (display coord) (newline)
            (let ([result (eval-cell coord grid '())])
              (display "Result: ") (display result) (newline)
              (cons coord result))))
        grid))
```

And trace `eval-cell` and `lookup` too:

racket

```
(define (lookup coord grid)
  (display "Looking up: ") (display coord) (newline)
  (let ([entry (assoc coord grid)])
    (display "Found: ") (display entry) (newline)
    (if entry
        (cdr entry)
        0)))

(define (eval-cell coord grid visited)
  (display "eval-cell coord: ") (display coord) (newline)
  (display "visited: ") (display visited) (newline)
  (cond
    [(member coord visited) '|#CYCLE|]
    [else
     (let ([value (lookup coord grid)])
       (display "value: ") (display value) (newline)
       (if (number? value)
           value
           (let ([result (eval-formula value grid (cons coord
visited) coord)])
             (cond
               [(eq? result '|#CYCLE|)  'cycle-error]
               [(eq? result '|#PARITY|) 'parity-error]
               [else result])))))]))
```

Run it and paste the output here — the trace will show exactly which coord and value is causing the hang, and we can pinpoint whether the issue is in `lookup`, `eval-formula`, or `resolve`.

a formula should be represented as a list after parsing

5:01 PM

That means `parse-entry` should use `list` instead of `cons` so the formula is kept as a list, and `lookup` should use `cadr` instead of `cdr`:

racket

```
; parse-entry : raw-entry -> (coord value)
```

```
(define (parse-entry e)
  (list (parse-coord (car e))
        (parse-value (cadr e))))

; lookup : coord grid -> number or formula
(define (lookup coord grid)
  (let ([entry (assoc coord grid)])
    (if entry
        (cadr entry)   ; cadr instead of cdr since entry is a list
        0)))
```

So the grid now looks like:

racket

```
; my-grid =>
'(((1 . 1) 10)
  ((1 . 2) (= (1 . 1) + 0))
  ((2 . 2) (= (1 . 2) * (1 . 1))))
```

And `eval-grid` uses `cadr` too when reconstructing entries:

racket

```
(define (eval-grid grid)
  (map (lambda (entry)
         (list (car entry)
               (eval-cell (car entry) grid '())))
       grid))
```

Each entry is now a two-element list `(coord value)` rather than a dotted pair `(coord . value)`, which means `assoc` still works correctly since it matches on `car`, but value access uses `cadr` throughout.

what exactly is being compared in the even checks for apply-op

6:58 PM

In `apply-op`:

racket

```
(define (apply-op op l r)
  (cond
    [(and (even? l) (odd? r)) '|#PARITY|]
    [(and (even? r) (odd? l)) '|#PARITY|]
    ...))
```

`l` and `r` are the resolved numeric values of the two terms — meaning they are the actual numbers after all cell lookups, distance taxes, and recursive evaluations have already happened. So for example with `(= (1 . 2) * (1 . 1))`:

racket

```
; (1 . 2) evaluates to 20, distance tax from (2 . 2) = 1, so l = 21
; (1 . 1) evaluates to 10, distance tax from (2 . 2) = 2, so r = 12
; (even? 21) = #f, (odd? 12) = #f -> no parity error
; (* 21 12) = 252
```

This means the parity check is comparing the post-tax values, not the raw cell values. If that's not the intended behavior and you want to check parity on the raw cell values before the distance tax is applied, the check would need to move into `resolve` before the tax is added:

racket

```
(define (resolve term grid visited current-coord)
  (cond
    [(number? term) term]
    [(pair? term)
     (let ([term-value (eval-cell term grid visited)])
       (cond
         [(eq? term-value '|#CYCLE|)  '|#CYCLE|]
         [(eq? term-value '|#PARITY|) '|#PARITY|]
         [else (+ term-value (manhattan-distance current-coord
term))]))]
    [else (error "Unknown term" term)]))
```

So the question for your spec is: should parity be checked on the raw cell value before the distance tax, or on the taxed value after? That determines which location is correct for the check.

project stipulations are: The Parity Check: If a cell's (x, y) coordinates sum to an even number, it can only reference other "even" cells. If it tries to reference an "odd" cell,
it returns a 'parity-error.

7:00 PM

So the parity check is based on the coordinate sum, not the cell's value. A cell is "even" or "odd" based on whether `(+ x y)` is even or odd:

racket

```
; coord-parity : coord -> boolean
(define (coord-even? coord)
  (even? (+ (car coord) (cdr coord))))
```

So the check belongs in `resolve`, where you know both the current cell's coord and the referenced cell's coord:

racket

```
(define (resolve term grid visited current-coord)
  (cond
    [(number? term) term]
    [(pair? term)
```

```
      (let ([term-value (eval-cell term grid visited)])
        (cond
          [(eq? term-value '|#CYCLE|)  '|#CYCLE|]
          [(eq? term-value '|#PARITY|) '|#PARITY|]
          ; if current coord is even but referenced coord is odd ->
parity error
          [(and (coord-even? current-coord) (not (coord-even? term)))
'|#PARITY|]
          [else (+ term-value (manhattan-distance current-coord
term))]))]
     [else (error "Unknown term" term)]))
```

And `apply-op` goes back to just doing arithmetic with no parity checks:

racket

```
(define (apply-op op l r)
  (match op
    ['+ (+ l r)]
    ['- (- l r)]
    ['* (* l r)]
    ['/ (if (zero? r) (error "Division by zero") (/ l r))]
    [_  (error "Unknown operator" op)]))
```

So for your grid, `(1 . 2)` has coordinate sum `3` which is odd, and `(2 . 2)` has coordinate sum `4` which is even — meaning `(2 . 2)` cannot reference `(1 . 2)` without a parity error.

is there example raw data input for a deep decay test case

7:33 PM

Here's a deep chain test case where each cell references the previous one:

racket

```
(define deep-chain
  '(((1 1) 2)
    ((1 2) (= (1 1) + 0))
    ((1 3) (= (1 2) + 0))
    ((1 4) (= (1 3) + 0))
    ((1 5) (= (1 4) + 0))))
```

All coords have odd sums (1+1=2... wait, let me reconsider with parity in mind:

racket

```
; coord sums:
; (1 1) -> 2 even
; (1 2) -> 3 odd
; (1 3) -> 4 even
; (1 4) -> 5 odd
; (1 5) -> 6 even
```

So this chain will hit parity errors since even cells are referencing odd cells. A deep chain that avoids parity errors uses only even-sum coords:

racket

```
; all even-sum coords: (1 1)=2, (2 2)=4, (3 3)=6, (4 4)=8
(define deep-even-chain
  '(((1 1) 2)
    ((2 2) (= (1 1) + 0))
    ((3 3) (= (2 2) + 0))
    ((4 4) (= (3 3) + 0))))
```

And a cycle test across a deep chain:

racket

```
; (4 4) eventually references back to (1 1) which references (4 4) ->
cycle
(define deep-cycle
  '(((1 1) (= (4 4) + 0))
    ((2 2) (= (1 1) + 0))
    ((3 3) (= (2 2) + 0))
    ((4 4) (= (3 3) + 0))))
```

Expected results:

- `deep-even-chain` — each cell gets the value of the previous plus the distance tax
- `deep-cycle` — every cell returns `cycle-error` since they all depend on the cycle

reconsider this when the manhattan tax is supposed to be subtracted from each value

7:39 PM

You're right. The tax should be subtracted, not added. Looking at `deep-even-chain`:

racket

```
; (1 1) = 2
; (2 2) references (1 1): distance = |2-1| + |2-1| = 2, so 2 - 2 = 0
; (3 3) references (2 2): distance = |3-2| + |3-2| = 2, so 0 - 2 = -2
; (4 4) references (3 3): distance = |4-3| + |4-3| = 2, so -2 - 2 =
-4
```

The chain decays quickly to negative numbers, which makes the test less clean. A better deep decay test uses a large starting value:

racket

```
(define deep-decay
  '(((1 1) 100)
    ((2 2) (= (1 1) + 0))
    ((3 3) (= (2 2) + 0))
    ((4 4) (= (3 3) + 0))
    ((5 5) (= (4 4) + 0))))
```

```
; expected results:
; (1 1) = 100
; (2 2) = 100 - 2 = 98
; (3 3) = 98  - 2 = 96
; (4 4) = 96  - 2 = 94
; (5 5) = 94  - 2 = 92
```

And update `resolve` to subtract instead of add:

racket

```
(define (resolve term grid visited current-coord)
  (cond
    [(number? term) term]
    [(pair? term)
     (let ([term-value (eval-cell term grid visited)])
       (cond
         [(eq? term-value '|#CYCLE|)  '|#CYCLE|]
         [(eq? term-value '|#PARITY|) '|#PARITY|]
         [(and (coord-even? current-coord) (not (coord-even? term)))
'|#PARITY|]
         [else (- term-value (manhattan-distance current-coord
term))]))]  ; subtract
    [else (error "Unknown term" term)]))
```

the tax logic should recurse

7:42 PM

The tax should compound as it propagates through the chain. Each cell already receives a taxed value from the previous cell, and then gets taxed again by its own distance. So the recursion is already happening naturally through `eval-cell` — the fix is just making sure the starting value at `(1 1)` is large enough to survive the chain:

racket

```
(define deep-decay
  '(((1 1) 100)
    ((2 2) (= (1 1) + 0))
    ((3 3) (= (2 2) + 0))
    ((4 4) (= (3 3) + 0))
    ((5 5) (= (4 4) + 0))))

; expected results with compounding tax:
; (1 1) = 100
; (2 2) references (1 1): distance (2,2)->(1,1) = 2, so 100 - 2 = 98
; (3 3) references (2 2): value is already 98, distance (3,3)->(2,2)
= 2, so 98 - 2 = 96
```

```
; (4 4) references (3 3): value is already 96, distance (4,4)->(3,3)
= 2, so 96 - 2 = 94
; (5 5) references (4 4): value is already 94, distance (5,5)->(4,4)
= 2, so 94 - 2 = 92
```

The recursion is natural because `eval-cell` always returns the already-taxed value of a cell, so each step in the chain is operating on a value that has already been reduced by all previous taxes. No changes to `resolve` are needed — the compounding behavior falls out of the recursive structure automatically.